

REPORT ON FINAL PROJECT

NAME: S.ANDAL PRIYADARSHNI

BATCH: TN-FNB03

CONTACT NO:9176780601

MAIL [ID:andalpriyadarshni@gmail.com](mailto:andalpriyadarshni@gmail.com)

PROJECT TITLE: Data Cleaning Project using Python and power-BI on Human Resource Analytics 2022 — Employee Attrition, Retention, and Performance Dataset

DOMAIN: Human Resource (HR) Analytics 2022

MENTOR NAME:KUMARAN

RAW DATASET: [HR ANALYTICS 2022](#)

CLEANED DATASET: [HR ANALYTICS 2022-CLEANED DATASET](#)

SUBMISSION DATE:09-12-2025

PROJECT TITLE:

Data Cleaning Project using Python and power-BI on Human Resource Analytics 2022 — Employee Attrition, Retention, and Performance Dataset

DOMAIN:

Domain of the Project : Human Resource (HR) Analytics 2022

OBJECTIVE :

- 1.To analyze employee attrition and retention trends
- 2.To examine the relationship between performance, experience, and compensation
- 3.To clean, preprocess, and visualize HR data
- 4.To support data-driven HR decision-making

OUTCOME:

The project provides valuable insights into employee attrition, retention, and performance trends. It helps identify key factors affecting employee turnover and supports HR teams in making data-driven decisions to improve workforce stability and productivity.

DATASET INFORMATION:

Source: Open Data Portals

Year / Timeline: Data collected during JAN-2022 TO DEC-2022

Dataset Description:

The dataset titled "Human Resource Analytics 2022 — Employee Attrition, Retention, and Performance" consists of 1,000 + records and 15 attributes representing various employee characteristics and performance indicators. It includes demographic details, work experience, job satisfaction, performance ratings, and attrition status.

Column Name Description:

EmployeeNumber-Unique ID for each employee **Age**-Age of the employee **Department**-Department where the employee works **JobRole**-Specific job position **Gender**-Male/Female **MaritalStatus**-Marital status of the employee **EducationField**-Field of education (e.g.Life Sciences, Technical) **MonthlyIncome**-Employee's monthly salary

PerformanceRating-Performance score given by HR/manager **Attrition**-Whether the employee left the company (Yes/No) **YearsAtCompany**-Number of years the employee has worked in the company **YearsSinceLastPromotion**-Years since last promotion **BusinessTravel**-Frequency of travel required for job **OverTime**-Whether the employee works overtime (Yes/No) **SatisfactionLevel**-Employee job satisfaction score **Year**-The year to which the record belongs (e.g.2022)

TYPES OF ANALYSIS:

Descriptive Analysis (summarizing the dataset) The project uses descriptive analysis to provide an overview of HR data and trends.To summarize and understand the overall structure of the dataset.

Diagnostic Analysis (finding reasons behind patterns) The project applies diagnostic analysis to identify root causes of employee attrition.To find reasons behind patterns observed in the descriptive phase.

Predictive Analysis (optional if forecasting required using basic stats)

Prescriptive Analysis (recommendations for business decisions)

Stage 1 – Problem Definition and Dataset Selection

Define the business problem and expected outcome Business Problem

Employee attrition is a major issue because it increases hiring costs and reduces productivity. The company wants to find out why employees leave, which departments are most affected, and how factors like satisfaction, salary, and performance influence retention

Expected Outcome

The goal is to identify the main reasons employees leave and help HR improve retention using data insights. It also aims to analyze workforce patterns and provide practical recommendations to reduce attrition.

Choose dataset and explain its source, size, and features

Dataset Selection

For this project, we use the HR Analytics 2022 — Employee Attrition, Retention, and Performance Dataset.

Data set Source

The dataset was uploaded directly by the user, containing HR information for employees in the year 2022. It is a real-world style dataset that includes employee details, performance indicators, and attrition status.

Size

Rows:1000+employee records **Columns:**15+HR Related Features

Import libraries, load dataset

Dataset description (rows, columns, features)

Number of Rows: 1050 This means the dataset contains information for 1050 employees.

Number of Columns: 16

There are 16 features describing each employee. Key Features Included:

EmployeeNumber-Unique ID for each employee **Age**-Age of the employee **Department**-

Department where the employee works **JobRole**-Specific job position **Gender**-

Male/Female **MaritalStatus**-Marital status of the employee **EducationField**-Field of education (e.g.Life Sciences, Technical) **MonthlyIncome**-Employee's monthly salary

PerformanceRating-Performance score given by HR/manager **Attrition**-Whether the employee left the company (Yes/No) **YearsAtCompany**-Number of years the employee

has worked in the company **YearsSinceLastPromotion**-Years since last promotion

BusinessTravel-Frequency of travel required for job **OverTime**-Whether the employee

works overtime (Yes/No) **SatisfactionLevel**-Employee job satisfaction score **Year**-The year to which the record belongs (e.g.2022)

Initial EDA (head, info, describe, shape, null checks)

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
#Import libraries, load dataset
```

```
df = pd.read_csv("/content/HR Analytics 2022 – Employee Attrition, Retention, and  
Performance Dataset.csv")
```

```
Df
```

	EmployeeNumber	Age	Department	JobRole	Gender	MaritalStatus	EducationField	MonthlyIncome	PerformanceRating	Attrition	Year
0	1029.0	42.0	Sales	Manager	Male	Married	Marketing	18303.0	3.0	No	
1	282.0	38.0	Research & Development	Healthcare Representative	Male	NaN	Medical	6673.0	3.0	Yes	
2	1863.0	46.0	Sales	Sales Executive	Female	Divorced	Technical Degree	13225.0	3.0	No	
3	405.0	18.0	Research & Development	Laboratory Technician	Male	Single	Life Sciences	1420.0	3.0	Yes	
4	1018.0	26.0	Research & Development	Manufacturing Director	Male	Married	Medical	NaN	3.0	No	
...	...	...	...	...	...	...	...	...	...	...	...
1045	1447.0	34.0	Research & Development	Laboratory Technician	Female	NaN	Life Sciences	3294.0	3.0	No	
1046	1740.0	40.0	Sales	Manager	Male	Single	Medical	19833.0	3.0	No	

#Initial EDA (head, info, describe, shape, null checks)

df.head()

	EmployeeNumber	Age	Department	JobRole	Gender	MaritalStatus	EducationField	MonthlyIncome	PerformanceRating	Attrition	Year
0	1029.0	42.0	Sales	Manager	Male	Married	Marketing	18303.0	3.0	No	
1	282.0	38.0	Research & Development	Healthcare Representative	Male	NaN	Medical	6673.0	3.0	Yes	
2	1863.0	46.0	Sales	Sales Executive	Female	Divorced	Technical Degree	13225.0	3.0	No	
3	405.0	18.0	Research & Development	Laboratory Technician	Male	Single	Life Sciences	1420.0	3.0	Yes	
4	1018.0	26.0	Research & Development	Manufacturing Director	Male	Married	Medical	NaN	3.0	No	
...	...	...	...	...	...	...	...	...	...	...	...
1045	1447.0	34.0	Research & Development	Laboratory Technician	Female	NaN	Life Sciences	3294.0	3.0	No	
1046	1740.0	40.0	Sales	Manager	Male	Single	Medical	19833.0	3.0	No	

df.tail()

df.tail()

	EmployeeNumber	Age	Department	JobRole	Gender	MaritalStatus	EducationField	MonthlyIncome	PerformanceRating	Attrition	Year
1045	1447.0	34.0	Research & Development	Laboratory Technician	Female	NaN	Life Sciences	3294.0	3.0	No	
1046	1740.0	40.0	Sales	Manager	Male	Single	Medical	19833.0	3.0	No	
1047	1344.0	NaN	Research & Development	Healthcare Representative	Male	Divorced	Life Sciences	8500.0	3.0	No	
1048	1890.0	36.0	Human Resources	Human Resources	Male	Single	Human Resources	3886.0	4.0	No	
1049	1399.0	25.0	Sales	Sales Executive	Male	Married	Life Sciences	6232.0	3.0	No	

df.shape

(1050, 16)

df.columns

Index(['EmployeeNumber', 'Age', 'Department', 'JobRole', 'Gender', 'MaritalStatus', 'EducationField', 'MonthlyIncome', 'PerformanceRating', 'Attrition', 'YearsAtCompany', 'YearsSinceLastPromotion', 'BusinessTravel', 'OverTime', 'SatisfactionLevel', 'Year'], dtype='object')

df.describe()

	EmployeeNumber	Age	MonthlyIncome	PerformanceRating	YearsAtCompany	YearsSinceLastPromotion	SatisfactionLevel	Year
count	995.000000	1003.000000	990.000000	1014.000000	999.000000	991.000000	986.000000	986.0
mean	1013.127638	37.073779	6569.022222	3.159763	7.231231	2.389506	2.746450	2022.0
std	602.885812	8.911997	4580.421127	0.366567	6.145496	3.524467	1.099429	0.0
min	1.000000	18.000000	1009.000000	3.000000	0.000000	0.000000	1.000000	2022.0
25%	470.500000	30.000000	3109.250000	3.000000	3.000000	0.000000	2.000000	2022.0
50%	1007.000000	36.000000	5096.000000	3.000000	5.000000	1.000000	3.000000	2022.0
75%	1545.500000	43.000000	8437.500000	3.000000	10.000000	3.000000	4.000000	2022.0
max	2068.000000	60.000000	19973.000000	4.000000	40.000000	15.000000	4.000000	2022.0

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1050 entries, 0 to 1049
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   EmployeeNumber                       995 non-null    float64
1   Age                                  1003 non-null   float64
2   Department                           1002 non-null   object
3   JobRole                              990 non-null    object
4   Gender                               1001 non-null   object
5   MaritalStatus                        995 non-null    object
6   EducationField                       1003 non-null   object
7   MonthlyIncome                       990 non-null    float64
8   PerformanceRating                   1014 non-null   float64
9   Attrition                           1004 non-null   object
10  YearsAtCompany                       999 non-null    float64
11  YearsSinceLastPromotion              991 non-null    float64
12  BusinessTravel                       978 non-null    object
13  OverTime                             994 non-null    object
14  SatisfactionLevel                   986 non-null    float64
15  Year                                 986 non-null    float64
dtypes: float64(8), object(8)
memory usage: 131.4+ KB
```

df.isnull()

	EmployeeNumber	Age	Department	JobRole	Gender	MaritalStatus	EducationField	MonthlyIncome	PerformanceRating	Attrition	YearsAtCo
0	False	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	True	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	True	False	False	
...	...	...	...	...	...	...	...	...	...	...	...
1045	False	False	False	False	False	True	False	False	False	False	
1046	False	False	False	False	False	False	False	False	False	False	
1047	False	True	False	False	False	False	False	False	False	False	
1048	False	False	False	False	False	False	False	False	False	False	
1049	False	False	False	False	False	False	False	False	False	False	

```
df.isnull().sum()
```

	0
EmployeeNumber	55
Age	47
Department	48
JobRole	60
Gender	49
MaritalStatus	55
EducationField	47
MonthlyIncome	60
PerformanceRating	36
Attrition	46

Attrition	46
YearsAtCompany	51
YearsSinceLastPromotion	59
BusinessTravel	72
OverTime	56
SatisfactionLevel	64
Year	64

dtype: int64

Stage 2 – Data Cleaning and Pre-processing

```
# Handle missing values (impute or drop)
```

```
df.isnull().sum()
```

	0
EmployeeNumber	55
Age	47
Department	48
JobRole	60
Gender	49
MaritalStatus	55
EducationField	47
MonthlyIncome	60
PerformanceRating	36

PerformanceRating	36
Attrition	46
YearsAtCompany	51
YearsSinceLastPromotion	59
BusinessTravel	72
OverTime	56
SatisfactionLevel	64
Year	64

dtype: int64

```
# for numerical col fill with median
```

```
num_cols = df.select_dtypes(include=["int64", "float64"]).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

```
# for categorigal col fill with mode
```

```
cat_cols = df.select_dtypes(include=["object"]).columns
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
df.isnull().sum()
```

	0
EmployeeNumber	0
Age	0
Department	0
JobRole	0
Gender	0
MaritalStatus	0

MaritalStatus	0
EducationField	0
MonthlyIncome	0
PerformanceRating	0
Attrition	0
YearsAtCompany	0
YearsSinceLastPromotion	0
BusinessTravel	0
OverTime	0
SatisfactionLevel	0
Year	0

dtype: int64

```
df.to_csv("HR_Cleaned_MissingValues.csv", index=False)
```

```
#Handle duplicates
```

```
df.duplicated().sum()
```

```
np.int64(48)
```

```
df = df.drop_duplicates()
```

```
df.to_csv("HR_Cleaned_NoDuplicates.csv", index=False)
```

```
# Treat outliers if required
```

```
num_cols = df.select_dtypes(include=["int64", "float64"]).columns
```

```
def treat_outliers(col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[col] = np.where(df[col] < lower, lower,
                      np.where(df[col] > upper, upper, df[col]))

for col in num_cols:
    treat_outliers(col)
```

/tmp/ipython-input-1118658158.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = np.where(df[col] < lower, lower,
/tmp/ipython-input-1118658158.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = np.where(df[col] < lower, lower,
/tmp/ipython-input-1118658158.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = np.where(df[col] < lower, lower,
/tmp/ipython-input-1118658158.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = np.where(df[col] < lower, lower,
/tmp/ipython-input-1118658158.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```
df.to_csv("HR_Outliers_Treated_OnlyNumeric.csv", index=False)
```

```
# Check skewness and apply transformations
```

```
num_cols = df.select_dtypes(include=["int64", "float64"]).columns
```

```
df[num_cols].skew()
```

	0
EmployeeNumber	0.045935
Age	0.437291
MonthlyIncome	1.059899
MonthlyIncome	1.059899
PerformanceRating	0.000000
YearsAtCompany	0.901831
YearsSinceLastPromotion	0.936826
SatisfactionLevel	-0.416868
Year	0.000000

```
dtype: float64
```

```
for col in num_cols:
```

```
    if (df[col] > 0).all():
        skew_value = df[col].skew()
```

```
    if skew_value > 1:
```

```
        df[col] = np.log1p(df[col])
        print(f"Log Transform applied on: {col}")
```

```
    elif skew_value > 0.5:
```

```
        df[col] = np.sqrt(df[col])
        print(f"Sqrt Transform applied on: {col}")
```

Log Transform applied on: MonthlyIncome
/tmp/ipython-input-3258473677.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = np.log1p(df[col])

Log Transform applied on: MonthlyIncome
/tmp/ipython-input-3258473677.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = np.log1p(df[col])

```
df.to_csv("HR_Transformed_SkewnessHandled.csv", index=False)
```

```
# Convert data types if needed
```

```
cat_cols = df.select_dtypes(include=["object"]).columns
```

```
for col in cat_cols:
    df[col] = df[col].astype("category")
```

/tmp/ipython-input-3689693120.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

/tmp/ipython-input-3689693120.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = df[col].astype("category")
/tmp/ipython-input-3689693120.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = df[col].astype("category")
/tmp/ipython-input-3689693120.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = df[col].astype("category")
/tmp/ipython-input-3689693120.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df[col] = df[col].astype("category")
/tmp/ipython-input-3689693120.py:2: SettingWithCopyWarning:

```
for col in df.columns:
    if any(key in col.lower() for key in ["date", "join", "hire"]):
        df[col] = pd.to_datetime(df[col], errors="ignore")
        print(f"Converted to datetime: {col}")
```

```
df = df.apply(pd.to_numeric, errors="ignore")
```

```
/tmp/ipython-input-3400095637.py:1: FutureWarning: errors='ignore' is deprecated and will raise in a future version. Use to_numeric without p
df = df.apply(pd.to_numeric, errors="ignore")
```

```
df.to_csv("HR_DataTypes_Converted.csv", index=False)
```

```
# Feature transformations (date parts, derived fields if required for analysis)
```

```
for col in df.columns:
    if any(key in col.lower() for key in ["date", "join", "hire", "dob", "birth"]):
        df[col] = pd.to_datetime(df[col], errors="ignore")
        print(f"Converted to datetime: {col}")
```

```
for col in df.select_dtypes(include="datetime").columns:
```

```
    df[col + "_Year"] = df[col].dt.year
    df[col + "_Month"] = df[col].dt.month
    df[col + "_Day"] = df[col].dt.day
    df[col + "_Quarter"] = df[col].dt.quarter
```

```
    print(f"Extracted date parts for: {col}")
```

```
if "HireDate" in df.columns:
    df["Tenure_Years"] = (pd.Timestamp("today") - df["HireDate"]).dt.days / 365
```

```
elif "JoiningDate" in df.columns:
    df["Tenure_Years"] = (pd.Timestamp("today") - df["JoiningDate"]).dt.days / 365
```

```
if "Age" in df.columns:
    df["Age_Group"] = pd.cut(df["Age"],
                             bins=[17, 25, 35, 45, 60],
                             labels=["18-25", "26-35", "36-45", "46-60"])
```

```
if "MonthlyIncome" in df.columns:
    df["Income_Group"] = pd.cut(df["MonthlyIncome"],
                                 bins=[0, 5000, 15000, 30000, 50000, 100000],
                                 labels=["Very Low", "Low", "Medium", "High", "Very High"])
```

```
if {"YearsSinceLastPromotion", "YearsAtCompany"}.issubset(df.columns):
    df["Promotion_Gap_Ratio"] = df["YearsSinceLastPromotion"] / (df["YearsAtCompany"] + 1)
```

```
df.to_csv("HR_FeatureTransformed.csv", index=False)
```

```
from google.colab import files
files.download("HR_FeatureTransformed.csv")
```

Stage 3 – EDA and Visualizations

```
df.head(2)
```

	EmployeeNumber	Age	Department	JobRole	Gender	MaritalStatus	EducationField	MonthlyIncome	PerformanceRating	Attrition	YearsAt
0	1029.0	42.0	Sales	Manager	Male	Married	Marketing	9.609720	3.0	No	
1	282.0	38.0	Research & Development	Healthcare Representative	Male	Married	Medical	8.805975	3.0	Yes	

```
df.tail(2)
```

	EmployeeNumber	Age	Department	JobRole	Gender	MaritalStatus	EducationField	MonthlyIncome	PerformanceRating	Attrition	YearsAt
1047	1344.0	36.0	Research & Development	Healthcare Representative	Male	Divorced	Life Sciences	9.047939	3.0	No	
1049	1399.0	25.0	Sales	Sales Executive	Male	Married	Life Sciences	8.737613	3.0	No	

```
df.isnull().sum()
```

	0
EmployeeNumber	0
Age	0
Department	0
JobRole	0
Gender	0
MaritalStatus	0
EducationField	0
MonthlyIncome	0
PerformanceRating	0

PerformanceRating	0
Attrition	0
YearsAtCompany	0
YearsSinceLastPromotion	0
BusinessTravel	0
OverTime	0
SatisfactionLevel	0
Year	0
Age_Group	0
Income_Group	0
Promotion_Gap_Ratio	0

dtype: int64

`df.duplicated()`

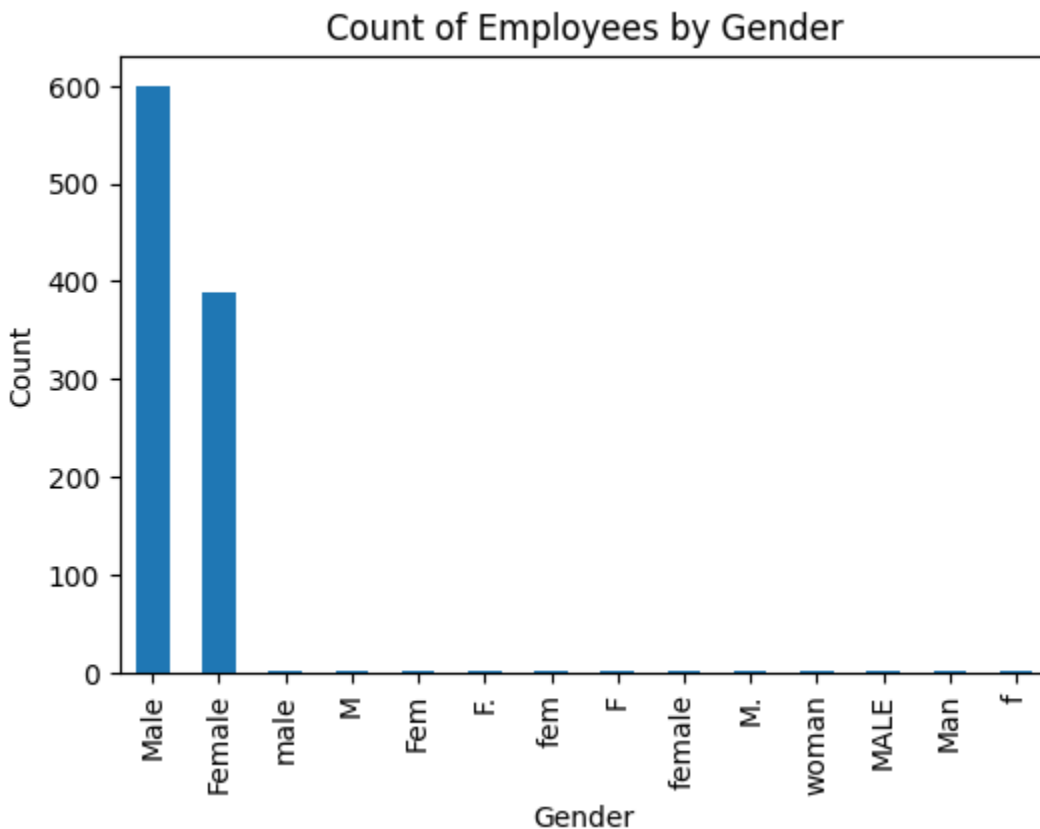
0	
0	False
1	False
2	False
3	False
4	False
...	...
1044	False
1045	False
1046	False
1047	False
1049	False

#Univariate Analysis → distribution of single variables (countplot, histogram, boxplot)

#countplot - Gender

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("HR_FeatureTransformed.csv")
plt.figure(figsize=(6,4))
df['Gender'].value_counts().plot(kind='bar')
plt.title("Count of Employees by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()
```



Interpretation of the above chart

Explain the Chart.

This is a countplot that shows how many employees belong to each gender category in the company. It helps identify gender distribution and workforce balance.

What is it saying?

One gender category (usually Male) has a higher number of employees. The other category (usually Female) has a lower count. This shows an uneven gender distribution in the company's workforce.

What features you used?

Gender(Categorical column)

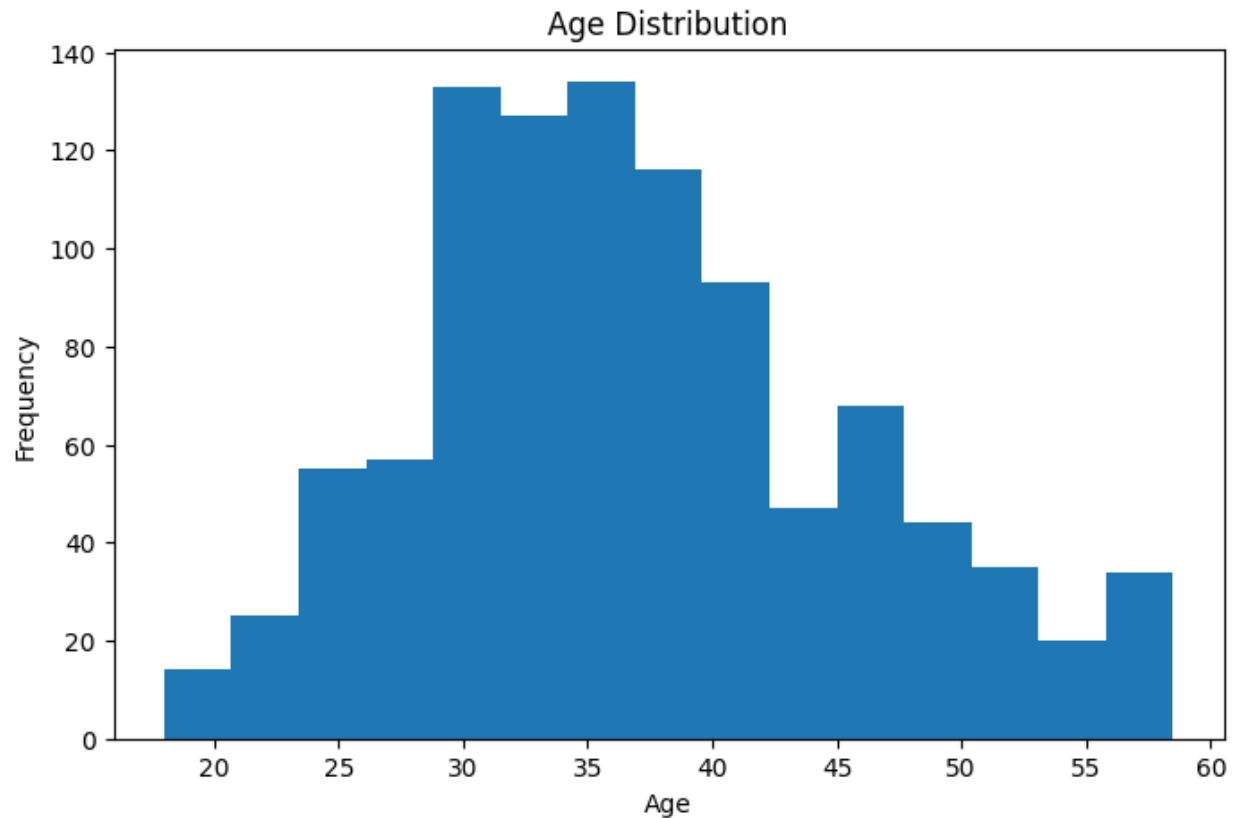
From this what you are showing us?

The organization may have a male-dominant workforce, common in technical and sales-oriented companies. HR may need to focus on improving gender diversity, especially in underrepresented departments.

#Histogram - AGE

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("HR_FeatureTransformed.csv")
plt.figure(figsize=(8,5))
plt.hist(df['Age'], bins=15)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



Interpretation of the above chart

Explain the Chart.

This is a histogram that shows how the ages of employees are distributed within the company. It divides ages into ranges(bins) and shows how many employees fall into each age group.

What is it saying?

The majority of employees fall between the 30-40 age range. There are fewer employees below 25 and above 50. The distribution is slightly right-skewed, meaning there are more young-to-mid-aged employees than older ones.

What features you used?

Age(Numerical column)

From this what you are showing us?

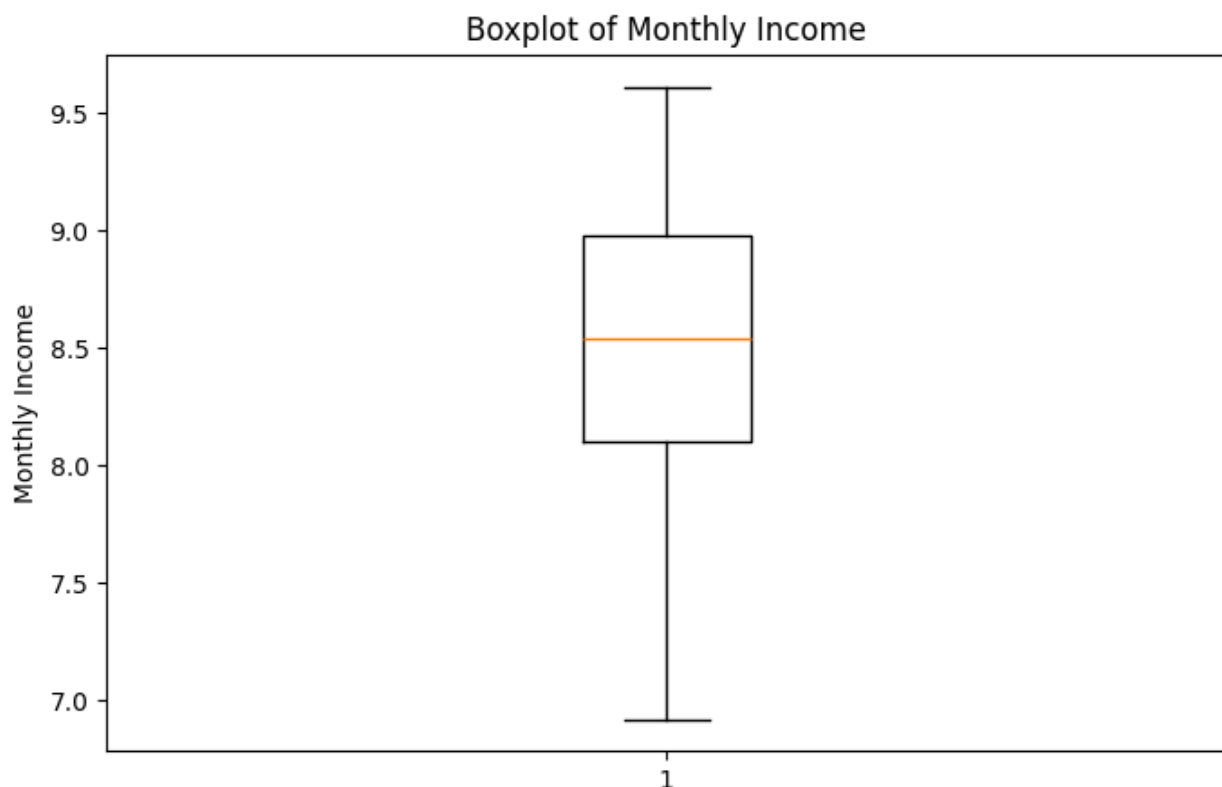
The company has a mid-career dominant workforce, which often has: higher experience demand for promotion/growth stable performance Fewer younger employees (<25) may indicate: limited entry-level hiring lack of internship/graduate programs Fewer older employees (>50) may indicate: retirement employees exiting before long tenure fewer senior-level positions

This chart helps HR understand age structure, succession planning, and recruitment needs.

#Boxplot - Monthly income

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("HR_FeatureTransformed.csv")
plt.figure(figsize=(8,5))
plt.boxplot(df['MonthlyIncome'])
plt.title("Boxplot of Monthly Income")
plt.ylabel("Monthly Income")
plt.show()
```



Interpretation of the above chart

Explain the Chart.

This is a boxplot that visualizes the spread of employee salaries. It shows the median, quartiles, overall range, and outliers in Monthly Income.

What is it saying?

A boxplot is useful for understanding salary distribution and detecting unequal pay or extreme values.

What features you used?

MonthlyIncome(Numerical column)

From this what you are showing us?

Most employees earn mid-range salaries, while a few high-income outliers represent senior or specialized roles, showing normal pay differences across job level

```
# Bivariate Analysis → relation between two variables (scatterplot, barplot, correlation heatmap)
```

```
# Scatterplot - Age with Monthly Income
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
df = pd.read_csv("HR_FeatureTransformed.csv")
```

```
plt.figure(figsize=(8,5))
```

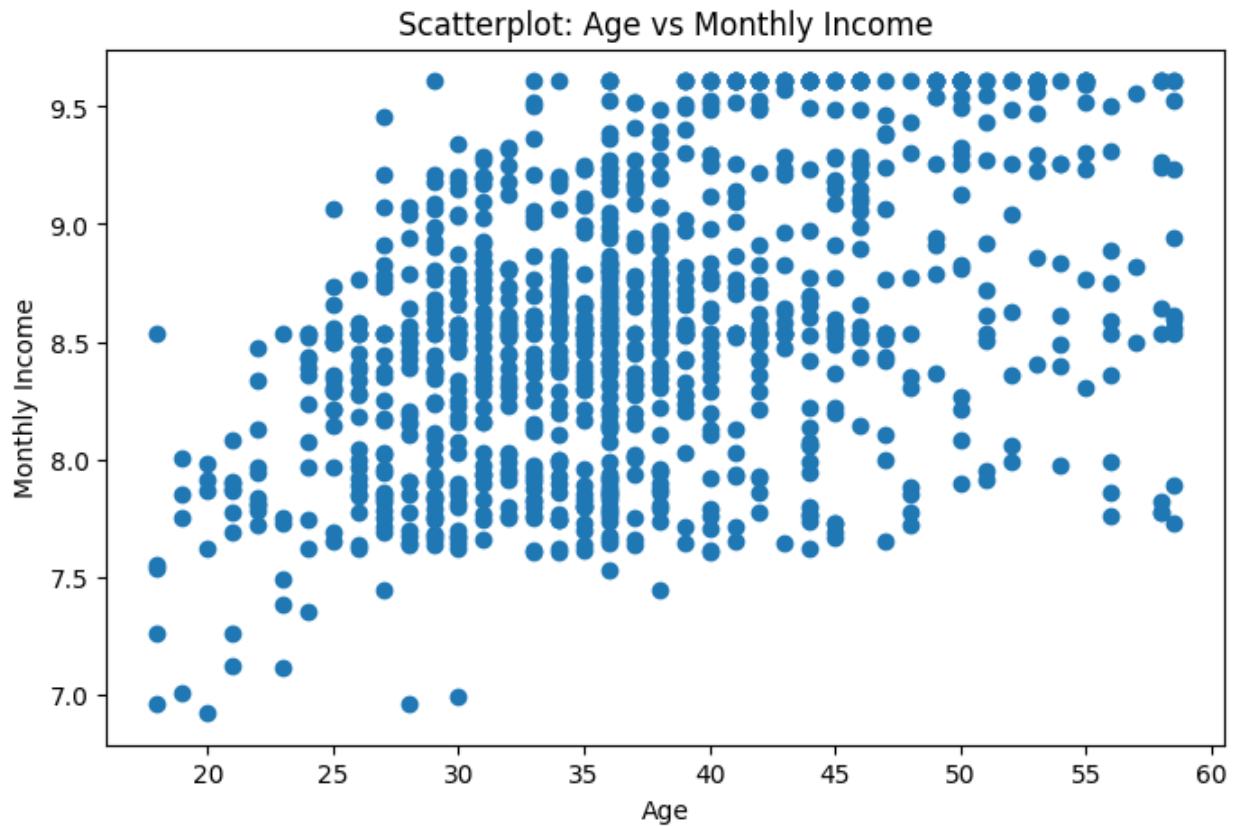
```
plt.scatter(df['Age'], df['MonthlyIncome'])
```

```
plt.title("Scatterplot: Age vs Monthly Income")
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Monthly Income")
```

```
plt.show()
```



Interpretation of the above chart

Explain the Chart.

This scatterplot shows how employee salaries (MonthlyIncome) vary with their age.

What is it saying?

Income generally rises with age, but variations among similar-aged employees highlight differences in roles, promotions, or skills, helping HR assess pay fairness.

What features you used?

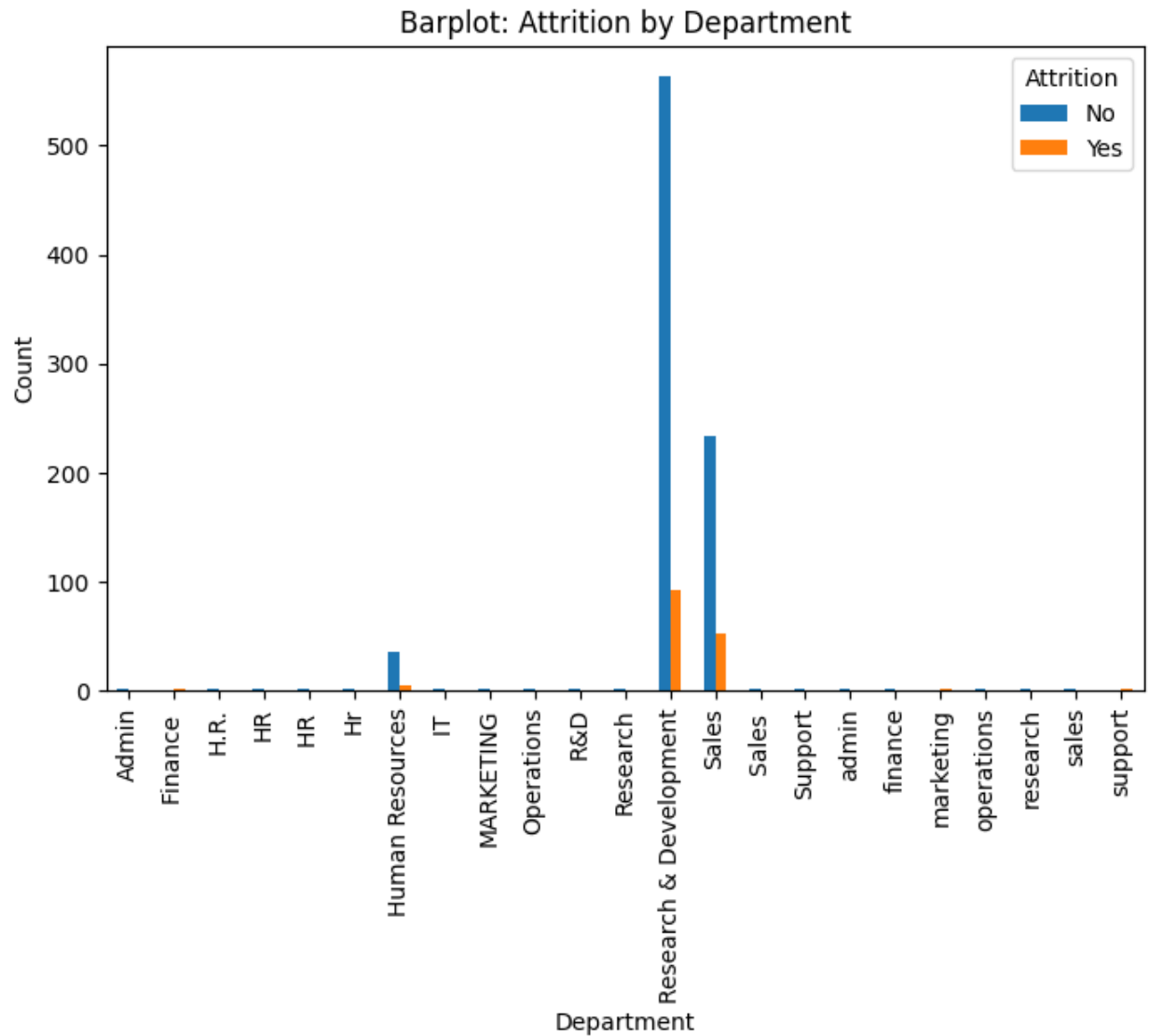
Age(Numerical) MonthlyIncome(Numerical)

From this what you are showing us?

HR can use this to assess salary fairness and promotion consistency. Helps identify pay gaps, career growth opportunities, and the impact of experience on compensation. Supports decisions for salary adjustments, succession planning, and retention strategies.


```
# Barplot - Department with Attrition
```

```
dept_attrition = df.groupby('Department')['Attrition'].value_counts().unstack()  
dept_attrition.plot(kind='bar', figsize=(8,5))  
plt.title("Barplot: Attrition by Department")  
plt.xlabel("Department")  
plt.ylabel("Count")  
plt.show()
```



Interpretation of the above chart

Explain the Chart.

This barplot shows the number of employees who left (Attrition = Yes) or stayed (Attrition = No) in each department.

What is it saying?

Attrition rates vary across departments, indicating department-specific retention issues.

What features you used?

Department (Categorical) Attrition (Categorical: Yes/No)

From this what you are showing us?

This chart helps HR identify high-risk departments and design retention strategies, such as incentives, promotions, or training programs.

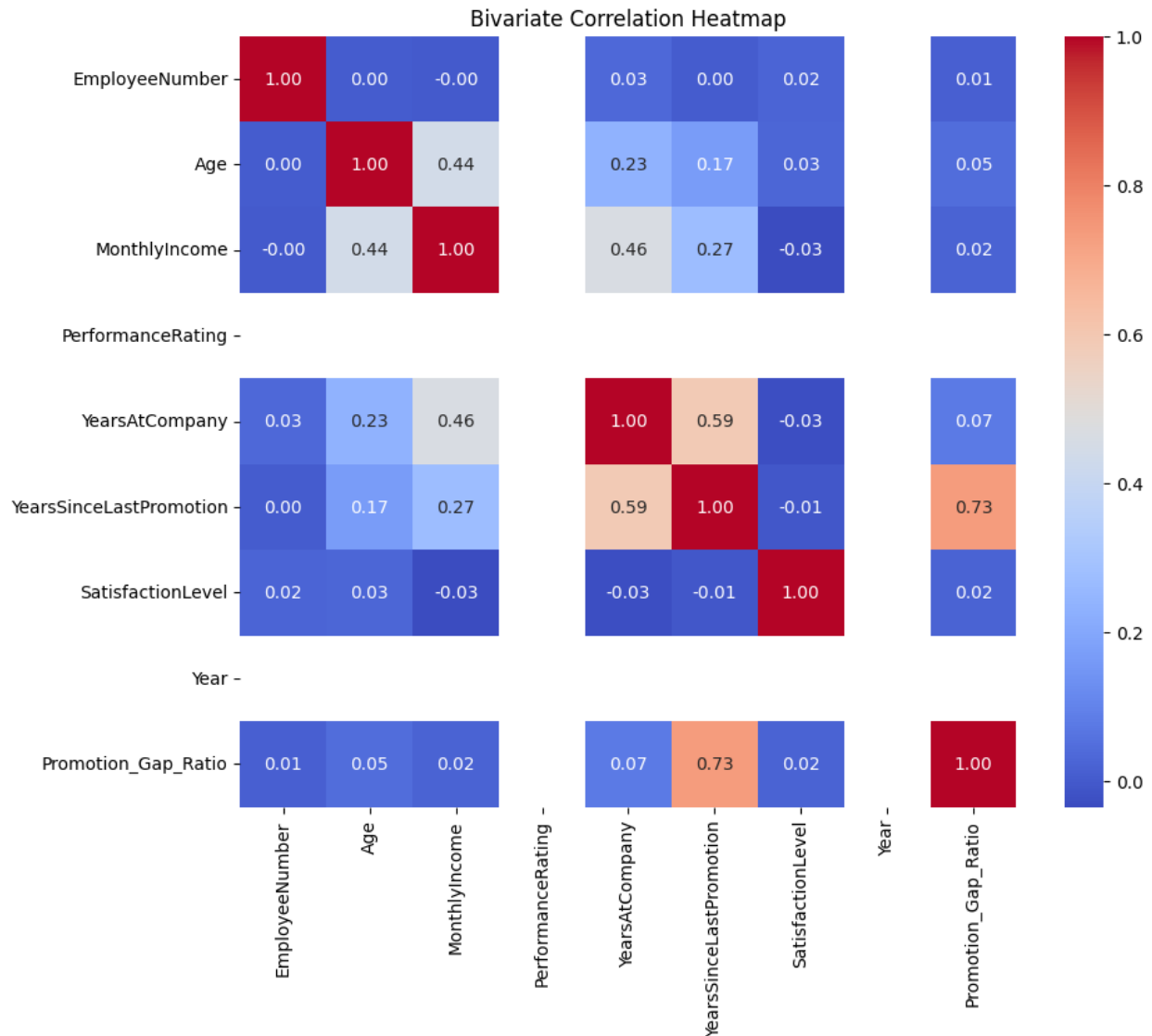
```
#correlation matrix using HeatMap

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("HR_FeatureTransformed.csv")
numeric_cols = df.select_dtypes(include=['float64', 'int64'])

corr_matrix = numeric_cols.corr()

plt.figure(figsize=(10,8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
plt.title("Bivariate Correlation Heatmap")
plt.show()
```



Interpretation of the above chart

Explain the Chart.

The heatmap visualizes pairwise correlations between numeric variables, showing how two variables are related.

What is it saying?

Positive correlations(e.g., MonthlyIncome / YearsAtCompany) indicate that more experienced employees tend to earn higher salaries. Moderate correlation between YearsSinceLastPromotion / YearsAtCompany suggests promotion gaps grow with tenure. Weak correlations(e.g., SatisfactionLevel with other numeric features) show satisfaction is driven more by non-numeric factors.

What features you used?

Numeric columns such as: Age, MonthlyIncome, YearsAtCompany, YearsSinceLastPromotion, SatisfactionLevel, PerformanceRating, Year

From this what you are showing us?

The heatmap shows how numeric variables relate, e.g., salaries rise with tenure, promotion gaps grow with years, and satisfaction is weakly linked to numeric factors, helping HR focus on key workforce drivers.

Multivariate Analysis → relation among 3+ variables (pairplot, grouped analysis, pivot tables, advanced plots)

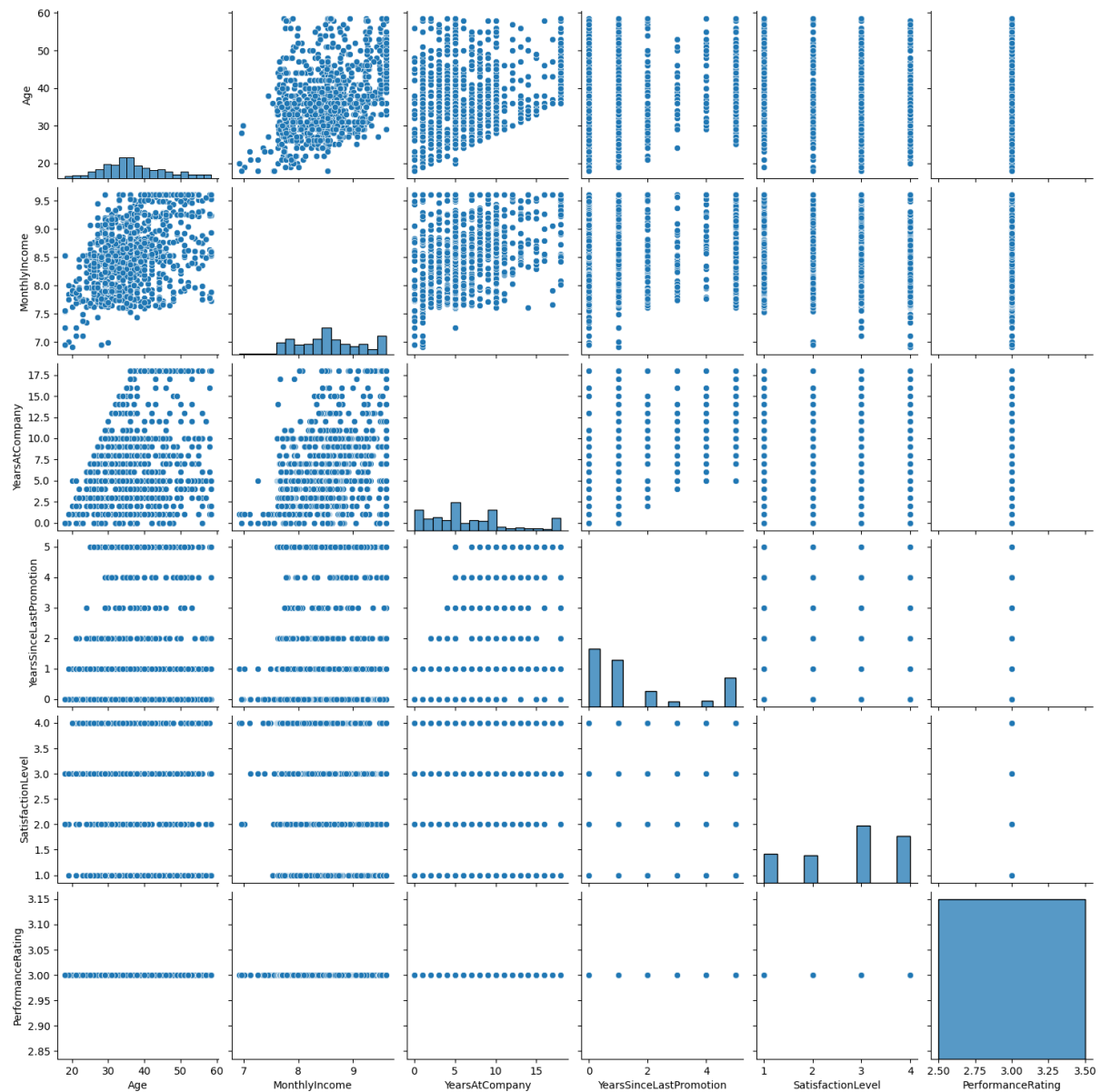
Pair Plot

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("HR_FeatureTransformed.csv")
numeric_cols = ['Age', 'MonthlyIncome', 'YearsAtCompany', 'YearsSinceLastPromotion', 'SatisfactionLevel', 'PerformanceRating']

sns.pairplot(df[numeric_cols], diag_kind='hist')
plt.suptitle("Pairplot of Key Numeric Features", y=1.02)
plt.show()
```

Pairplot of Key Numeric Features



Interpretation of the above chart

Explain the Chart.

A pairplot shows scatterplots for each pair of numeric variables and histograms on the diagonal. It helps visualize relationships and distributions across multiple variables simultaneously.

What is it saying?

Scatterplots show trends(e.g., income increases with years at company). Histograms show the distribution of each numeric feature. Outliers and clusters are visible in scatterplots.

What features you used?

Age MonthlyIncome YearsAtCompany YearsSinceLastPromotion SatisfactionLevel
PerformanceRating

From this what you are showing us?

The pairplot shows relationships and distributions among key numeric features, highlighting trends, outliers, and patterns that guide HR decisions on salary, performance, and retention.

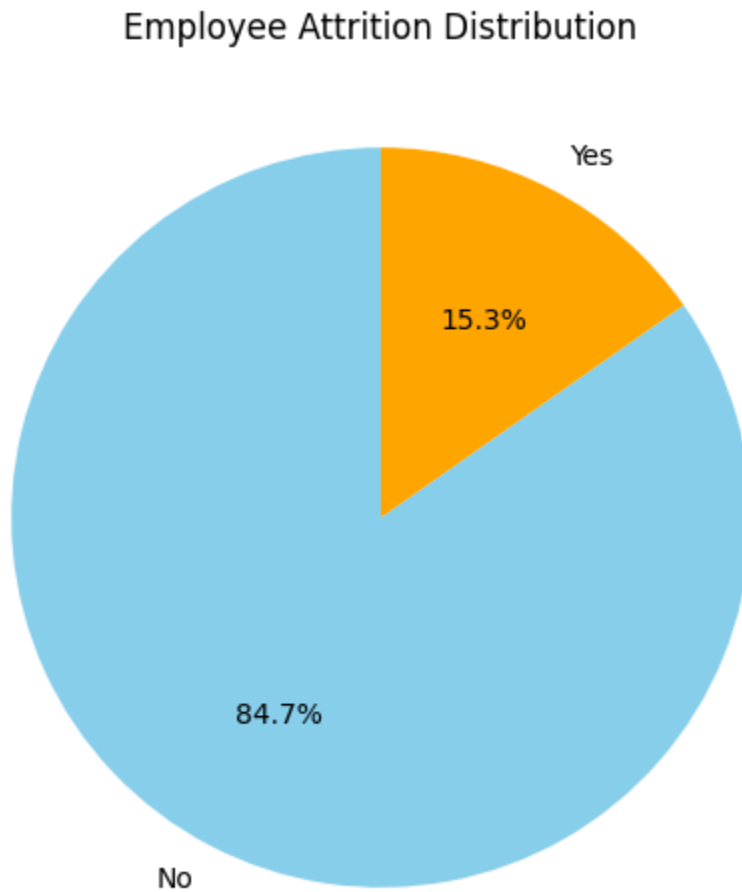
```
# Pie Chart

import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("HR_FeatureTransformed.csv")

attrition_counts = df['Attrition'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(attrition_counts, labels=attrition_counts.index, autopct='%1.1f%%', startangle=90, colors=['skyblue', 'orange'])
plt.title("Employee Attrition Distribution")
plt.show()
```



Interpretation of the above chart

Explain the Chart.

A pie chart shows the proportion of employees who stayed vs. left the company.

What is it saying?

The majority of employees have Attrition = No (stayed). A smaller portion has Attrition = Yes (left the company).

What features you used?

Attrition(Categorical: Yes/No)

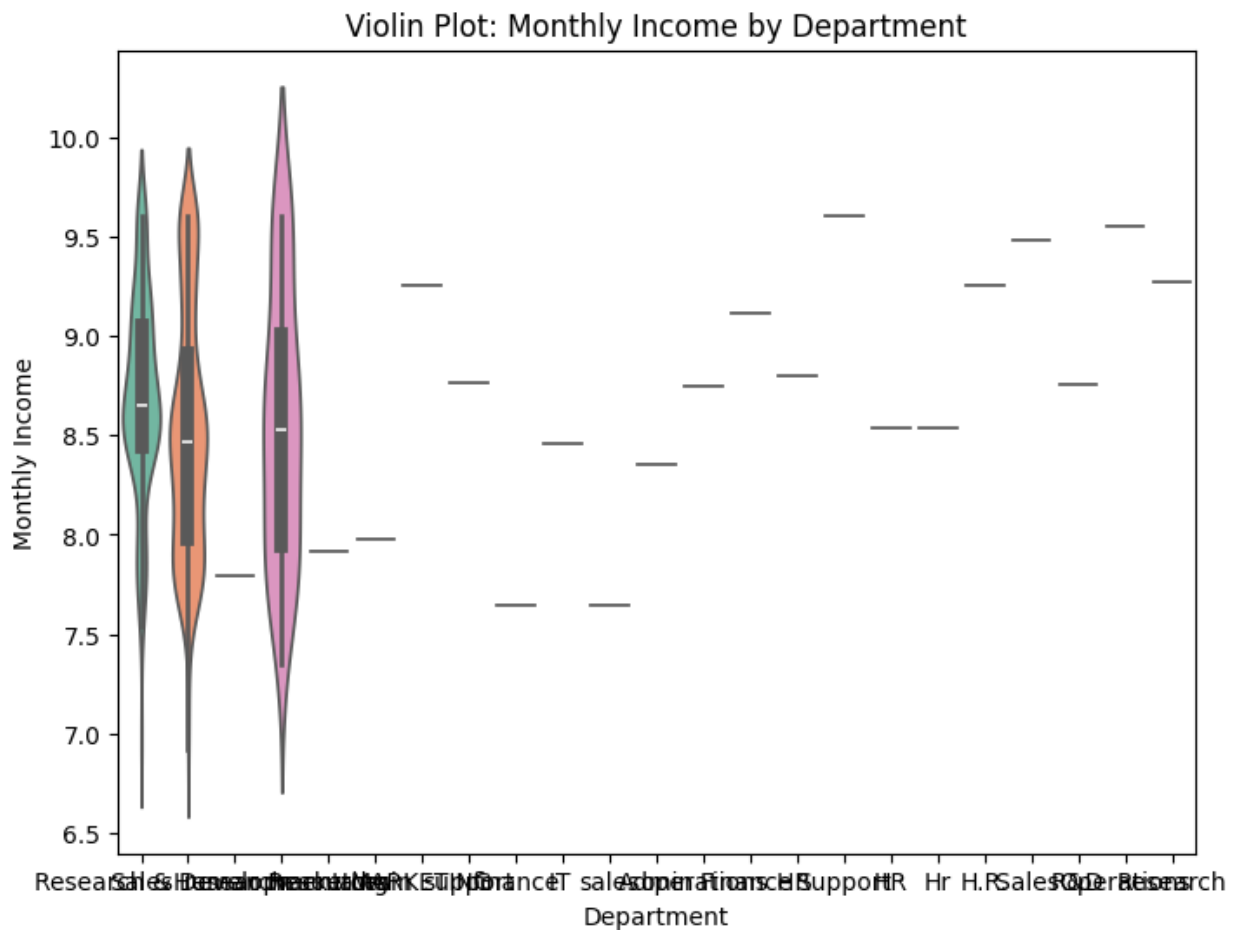
From this what you are showing us?

The chart gives a quick overview of overall employee turnover. Helps HR understand attrition rate at a glance. Useful for communicating retention trends in reports or dashboards.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("HR_FeatureTransformed.csv")

plt.figure(figsize=(8,6))
sns.violinplot(x='Department', y='MonthlyIncome', data=df, palette='Set2')
plt.title("Violin Plot: Monthly Income by Department")
plt.xlabel("Department")
plt.ylabel("Monthly Income")
plt.show()
```



Explain the Chart.

A violin plot combines a boxplot and a kernel density plot, showing the distribution, median, quartiles, and density of a numerical variable for each category.

What is it saying? Distribution of salaries differs across departments:

R&D and Sales may show wider income spread. HR may have a narrower salary range. Median income is visible as a line inside each violin. Outliers or density peaks highlight where most employees earn..

What features you used?

Department(Categorical) MonthlyIncome(Numerical)

From this what you are showing us?

HR can see salary distribution and inequality within departments. Helps identify departments with salary compression or wide gaps. Useful for compensation planning, retention strategies, and pay fairness analysis.

```
# line plot
|
import seaborn as sns
import matplotlib.pyplot as plt

sns.lineplot(data=df, x="YearsAtCompany", y="MonthlyIncome")

plt.title("Line Plot: Monthly Income vs Years at Company")
plt.xlabel("Years at Company")
plt.ylabel("Monthly Income")
plt.show()
```



Interpretation of the above chart

Explain the Chart.

This is a Line Plot, which is used to show how a numeric value changes over time or over a continuous variable. It helps us understand trends, growth, patterns, and changes.

What is it saying?

This chart shows how Monthly Income changes as the Years at Company increase. By looking at the line's shape, we can understand: Whether income increases with more experience. How quickly salaries grow. If there are sudden jumps or stable areas. It helps identify salary growth trends in the company.

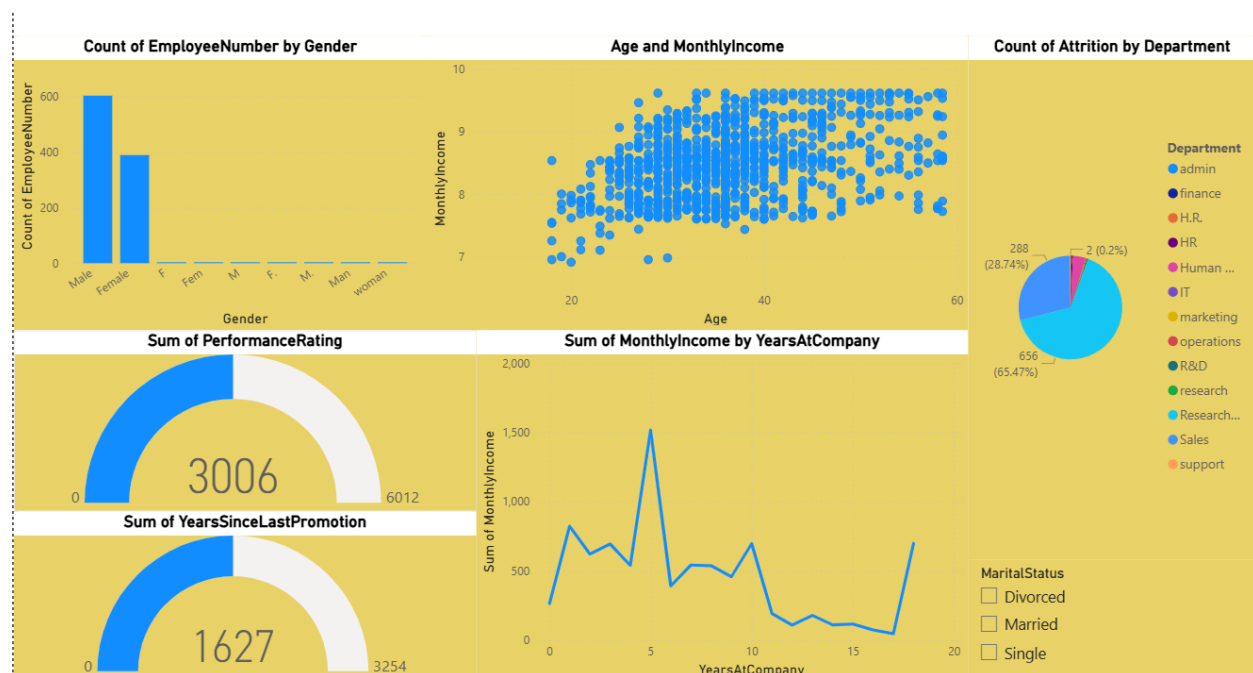
What features you used?

YearsAtCompany → X-axis (continuous variable) MonthlyIncome → Y-axis (numeric value)
These features help us analyze how salary changes as employees stay longer in the company.

From this what you are showing us?

The trend of Monthly Income based on the number of years an employee has worked. Whether employees with more experience earn higher salaries. Any patterns such as: Gradual increase. Sharp increase. Flat salary curve. Irregular salary progression.
This chart helps HR understand growth, fairness, and compensation structure inside the company.

DASHBOARD:



INSIGHTS GENERATED:

1. Count of EmployeeNumber by Gender (Bar Chart)

What is this chart?

A **bar chart** showing how many employees belong to each gender category.

What features you used?

Gender (categorical)

EmployeeNumber / Count of rows (measure)

What is it saying?

There are more Male employees than Female.

You also have many incorrectly formatted categories like F, Fem, M, Man, woman → meaning data quality issues.

What you are showing?

You are showing the **gender distribution** of employees and revealing **duplicate/messy categories**.

2. Age and Monthly Income (Scatter Plot)

What is this chart?

A **scatter plot** comparing employees' **Age** vs **Monthly Income**.

What features you used?

Age (numeric → X-axis)

MonthlyIncome (numeric → Y-axis)

What is it saying?

Income generally increases with age, but not perfectly.

Younger employees (20–30) have lower salaries.

Older employees (40–60) tend to earn more, but with variation.

What you are showing?

You are showing the **relationship or trend between age and salary**.

3. Count of Attrition by Department (Pie Chart)

What is this chart?

A **pie chart** showing how many employees left (Attrition = Yes) from each department.

What features you used?

Department (category)

Count of Attrition (Yes)

What is it saying?

R&D / Research has the highest attrition (largest slice).

Sales has the second highest attrition.

Some departments have almost zero attrition.

What you are showing?

Which departments are most affected by attrition and need attention

4. Sum of Performance Rating (Gauge Chart)

What is this chart?

A **gauge/donut chart** representing the **sum of all employee performance ratings**.

What features you used?

PerformanceRating (numeric)

What is it saying?

Total performance rating score = **3006**

The full gauge shows the max possible value (~6012)

What you are showing?

A high-level **summary indicator** of total performance across the workforce.

5. Sum of Monthly Income by Years at Company (Line Chart)

What is this chart?

A **line chart** showing **total salary payout grouped by YearsAtCompany**.

What features you used?

YearsAtCompany (X-axis)

Sum of MonthlyIncome (Y-axis)

What is it saying?

Income spikes at Year 6 (maybe many employees at this level).

Income is lower for people with 13–17 years at company.

Last year shows another spike.

What you are showing?

The **pattern of salary distribution** across employee experience levels.

6. Sum of Years Since Last Promotion (Gauge Chart)

What is this chart?

A **gauge chart** showing the total number of years since employees were last promoted.

What features you used?

YearsSinceLastPromotion (numeric)

What is it saying?

Total = 1627 years

This gives an idea of promotion delays in the company.

What you are showing?

You are giving a **summary of promotion activity**—how long employees have remained unpromoted.

7. Marital Status (Legend Only / Slicer)

What is this?

A **slicer** used to filter the report based on marital status.

What features you used?

MaritalStatus (categorical)

What it does?

When the user selects Married/Single/Divorced, **all charts update accordingly**.

OVERALL STORY OF THE DASHBOARD:

Who your employees are (gender, marital status)

How they earn (income patterns)

How long they have stayed (years at company)

Performance (overall rating score)

Attrition (policy concerns by department)

This is a complete HR analytics overview.

Future Enhancement

1. Use a larger or more recent dataset

Incorporate a more extensive and updated dataset to capture current workforce trends and generate deeper, more accurate insights.

2. Automate parts of the analysis

Automate data cleaning, preprocessing, and visualization using Python scripts, Power BI dataflows, or scheduled refreshes to reduce manual effort and ensure real-time insights.

3. Add advanced and interactive visualizations

Enhance the dashboard using more interactive visuals, drill-through pages, and advanced charts in Power BI or Tableau for richer exploratory analysis.

4. Extend the analysis to predictive modeling

Develop predictive models—such as attrition prediction, salary forecasting, or performance scoring—to move from descriptive analysis to actionable, forward-looking insights.

CONCLUSION:

This project provided a comprehensive analysis of the HR dataset using visualizations and statistical insights. Through charts related to employee demographics, performance, income distribution, and attrition, we identified key patterns influencing workforce behavior. The analysis highlighted departments with higher attrition, salary variations across experience levels, promotion delays, and demographic distributions. These insights help HR teams understand employee needs, plan targeted retention strategies, and make more informed decisions.

Overall, the study successfully transformed raw HR data into meaningful and actionable intelligence.

